

passage-contract

Passage / PID Contract — Workshop Curriculum Platform

Feature: specs/001-workshop-curriculum-platform | **Phase:** 1 (Design & Contracts) | **Date:** 2026-09-01

Governs: passage identity — the internal contract every other component depends on. **Derived from:** [spec.md](#) FR-037/FR-038/FR-039, [research.md](#) D-SEARCH-1, [data-model.md](#) Passage. **Consumed by:** [http-api.md](#) ({pid} path parameter, citations, cross-references, 410 redacted) and [pipeline-cli.md](#) (ingest, index, crossref, redact).

This is the contract [plan.md](#) marks [REVIEW] — “*review before anything consumes it; every other component depends on its shape.*” It specifies shapes, schemas and guarantees. No implementation, no migrations.

1. The one rule

pid is a ULID minted once at ingest. It is not derived from content and it is not derived from position. Nothing else is ever a citable identity.

This is not a style preference. Both alternatives were **measured**, and both fail, in opposite directions (D-SEARCH-1, controlled three-run re-index):

Candidate identity	Measured behaviour	Consequence
Lumen chunk id (content-derived)	Fixing one typo inside a section, line numbers unchanged, changed the id	Every human correction (FR-038) silently breaks every

Candidate identity	Measured behaviour	Consequence
	4b98e295c112a429 → 610e608a50f273a6	citation to that passage
Positional key (file + line range)	Prepending a section shifted every later section by 4 lines: all ids unchanged, all line ranges moved	Every edit above a passage breaks its links

Under either, **SC-016 scores 0% on corrected passages while the UI keeps rendering links that now point somewhere else** — a failure with no visible symptom. Lumen’s ids are not exposed through its CLI or MCP surface anyway, leaving only a positional key, which run 3 shows moves. Both available keys fail.

A minted identifier is the only form that survives both operations. Everything below exists to keep that true.

2. The pid

2.1 Format

Property	Value
Type	ULID (Crockford base32)
Length	exactly 26 characters
Canonical case	UPPERCASE
Validation regex	<code>^[0-7][0-9A-HJKMNP-TV-Z]{25}\$</code>
Excluded letters	I, L, O, U (Crockford alphabet)
Serialised as	a bare string. Never wrapped, never prefixed, never namespaced.

Lowercase input is accepted at every boundary and canonicalised to uppercase before storage, comparison or response. Two pids are equal iff their canonical forms are byte-equal — no normalisation beyond case, no trimming of “similar-looking” characters.

2.2 Minting rules (normative)

#	Rule
M1	A pid is minted exactly once , at the first ingest that observes the passage.
M2	Only the ingest command mints. No other component — not the API, not the indexer, not the cross-referencer, not the frontend — may create a pid.
M3	A pid is never reused, never reassigned, and never recycled , including after the passage it named is deleted. Deletion tombstones the row; it does not free the identifier.
M4	A pid is minted only when the ingest matching algorithm (§8) fails to attach the passage to an existing registry row. Minting is the last branch, never the first.
M5	The ULID timestamp component is the ingest wall-clock. It carries no semantic meaning . Sorting passages by pid, inferring recording order from pid, or reading a date out of a pid is a defect. Monotonicity is guaranteed only within a single ingest process, and is an implementation convenience, not a contract.
M6	Minting is not idempotent by luck — it is idempotent by M4. Re-running ingest on unchanged input mints zero pids (FR-027).

2.3 What a pid is not

It is not an ordering key, not a version, not a hash, not a URL, not a file path, not a line number, not a Lumen chunk id, and not derivable from any of them. Given a pid and nothing else, the only way to learn anything about the passage is to ask the registry (§7).

3. Source anchors

The pid is written back into the source artifact so **the identifier travels with the text**. This is the mechanism that delivers guarantee G2 (§5): when a passage moves, its anchor moves with it, and the next ingest re-derives the position rather than re-deriving the identity.

3.1 The canonical anchor

```
<!-- pid: 01JBX7QK3M8V2ZC4YT5N6RWDPA -->
```

Property	Contract
Placement	On its own line, immediately preceding the passage's first line. No blank line between anchor and passage.
Recognition regex	<code>^<!--\s*pid:\s*([0-7][0-9A-HJKMNP-TV-Z]{25})\s*-->\$</code>
Emission form	Exactly <code><!-- pid: <PID> --></code> — one space after <code><!--</code> , one space either side of the pid, one space before <code>--></code> . Emission is canonical even though recognition is tolerant, so that re-ingest produces byte-identical files (FR-027).
Cardinality	One anchor per passage. A pid appears as an anchor at most once in the entire repository .
Rendering	HTML comments do not render in markdown output, so anchors are invisible to a reader while

Property	Contract
	remaining in the source a human edits.

3.2 Per-format anchor syntax

Markdown is the primary case. The others exist because “write a comment” is not universally possible, and pretending otherwise would be inventing capability.

Artifact kind	Anchor syntax	Notes
Markdown, HTML	<code><!-- pid: ... --></code>	Canonical.
Mermaid (.mmd)	<code>%% pid: ...</code>	Must precede the node/statement it identifies.
PlantUML (.puml)	<code>' pid: ...</code>	
SVG	<code><!-- pid: ... --></code> inside the root element	
WebVTT / SRT (.vtt, .srt)	none — sidecar	These formats have no comment syntax that survives every parser. The pid lives in <code><file>.pids.jsonl</code> , keyed by cue identifier.
Transcript chunk JSON	none — sidecar	Machine artifacts under <code>transcript/work/</code> are regenerable and must not be hand-edited; the pid lives in the registry, keyed by <code>(chunk_index, segment_index)</code> .
Source code	none — registry only	See §3.3.
PDF, images, audio, video	none — registry only	Binary; not editable by this pipeline (FR-006).

Sidecar records use the same shape as a registry line (§4.2) restricted to {pid, key}, where key is the format’s own stable within-file identifier.

3.3 Where anchors are never written

Rule	Reason
Never inside source code.	The D2 corpus is the whole vasic monorepo. Writing anchors into code the workshop does not own would modify unrelated submodules, pollute diffs, and violate the spirit of FR-006. Code passages are registry-only (§6).
Never outside workshop/.	Any passage whose source_ref.path resolves outside the workshop/ submodule is registry-only, and inherits the weaker guarantee of §6 — <i>including</i> markdown files elsewhere in the monorepo.
Never into a machine artifact under transcript/work/.	Those files are regenerable checkpoints; an anchor there would be destroyed on the next resume.
Never into the recording, the notes PDF, or any supplied material.	FR-006: source material is preserved unmodified. Ingest opens them O_RDONLY.

3.4 Anchor authority

- The **anchor is authoritative** for the text↔identity association.
- source_ref in the registry is a **cache** of where the anchor was last seen, and MUST be re-derived from the file on every ingest.
- A registry row whose source_ref disagrees with the file is not a conflict — the file wins, and ingest updates the row. That is guarantee G2 operating normally, not an error.
- An anchor in a file whose pid is **absent from the registry** is an error, not an invitation to mint (§8, rule R1c). Silently adopting a foreign anchor would let a copy-pasted block hijack an identity.

4. The registry

Two artifacts, one authoritative.

Artifact	Role	Tracked in git
workshop/curriculum/passages.jsonl	Source of truth. Human-reviewable, line-oriented, diffable.	Yes
workshop/curriculum/passages.db	Derived SQLite: query index, FTS5, join tables. Rebuildable from the JSONL at any time.	No (git-ignored)

The identity of the system lives in a text file a human can read in a pull request. The database is a cache. If they disagree, the JSONL wins and the database is rebuilt.

4.1 passages.jsonl file-level contract

Property	Contract	Reason
Encoding	UTF-8, NFC-normalised, LF line endings, final newline present	Byte-stable across platforms
One record per line	Compact JSON, no embedded newlines	git diff shows one changed passage as one changed line
Key order	Fixed, as listed in §4.2	Byte-identical output on re-ingest (FR-027)
Record order	Sorted ascending by pid	A minted pid lands deterministically; review diffs stay small
Whitespace	No trailing whitespace, no pretty-printing	
Idempotency	Re-running ingest on unchanged input	The mechanical test for FR-027

Property	Contract	Reason
	produces a byte-identical file	

4.2 Record schema

```

{
  "pid": "01JBX7QK3M8V2ZC4YT5N6RWDPA",
  "kind": "transcript",                                // transcript |
doc_section | code | diagram
  "chapter_slug": "01-ai-workflows",                    // null for corpus-wide
passages outside a chapter
  "source_ref": {
    "path": "chapters/01/transcript/transcript.md",    // repo-
relative, forward slashes
    "line_start": 812,
    "line_end": 815,
    "symbol": null,                                    // code kind only – see
§6
    "anchor": "inline"                                // "inline" | "sidecar"
| "registry_only"
  },
  "text": "we always cut inside a measured silence, never mid-
word",
  "machine_text": "we always cut inside a measured silence never
mid word",
                                                                // transcript kind
only; IMMUTABLE once written (FR-038)
  "provenance": "human_corrected",                      // machine |
human_corrected
  "confidence": -0.31,                                  // avg_logprob-derived;
null for non-transcript kinds
  "uncertain": false,
  "uncertain_reason": null,                             // low_confidence |
inaudible | overlapping_speech
                                                                // |
language_uncertain ; null iff uncertain==false
  "t_start_s": 512.4,                                   // transcript kind
only; null otherwise
  "t_end_s": 519.1,
  "speaker": "Milos",                                  // null unless human-
attributed
  "speaker_source": "human",                            // human |
unattributed (never "machine" – D-TRANS-2)
  "redacted": false,
  "content_hash": "sha256:8f1c...",                    // CHANGE DETECTION
ONLY – see §6bis / §5.3
  "minted_at": "2026-09-01T09:12:44Z",                 // provenance of the
identifier, not of the text
  "ingest_run": "01JBX7QJ..."                         // run id that minted

```



```
or last touched the row
}
```

Required fields: pid, kind, source_ref, text, provenance, redacted, content_hash, minted_at. All others are nullable with the constraints noted.

Field-level invariants

#	Invariant
F1	machine_text is written once and never mutated. A correction adds a corrections record and updates text; machine_text is untouched. Deleting it would destroy the ability to re-measure accuracy later (SC-002).
F2	provenance == "human_corrected" ⇒ machine_text is non-null and at least one corrections record exists for this pid.
F3	uncertain == true ⇔ uncertain_reason != null. Neither may be inferred downstream; uncertain is set from the engine's own confidence signal at ingest (FR-003).
F4	speaker != null ⇒ speaker_source == "human". speaker_source may never be "machine" — the audio is dual-mono (L-R = -90.3 dB) and AGC-flattened, so both diarization cues are measurably absent (D-TRANS-2).
F5	kind == "transcript" ⇒ t_start_s and t_end_s are non-null, and t_start_s < t_end_s.
F6	redacted == true ⇒ text and machine_text are absent from every serialisation that leaves the registry, at every layer (§7.3).

#	Invariant
F7	content_hash is present on every row and is never used as a key (§5.3).

4.3 passages.db schema (normative declaration, not a migration)

Derived. Rebuildable. modernc.org/sqlite, already a dependency of the reference module — no new dependency (D-SEARCH-2).

```
-- Identity. One row per passage, forever.
CREATE TABLE passages (
  pid          TEXT PRIMARY KEY,          -- canonical
           uppercase ULID
  kind         TEXT NOT NULL CHECK (kind IN
    ('transcript', 'doc_section', 'code', 'diagram')),
  chapter_slug TEXT,
  path         TEXT NOT NULL,
  line_start   INTEGER, line_end INTEGER,
  symbol       TEXT,                    -- code kind only
  anchor       TEXT NOT NULL CHECK (anchor IN
    ('inline', 'sidecar', 'registry_only')),
  text         TEXT,                    -- NULL when
           redacted
  machine_text TEXT,                    -- immutable; NULL
           when redacted
  provenance   TEXT NOT NULL CHECK (provenance IN
    ('machine', 'human_corrected')),
  confidence   REAL,
  uncertain    INTEGER NOT NULL DEFAULT 0,
  uncertain_reason TEXT,
  t_start_s    REAL, t_end_s REAL,
  speaker      TEXT,
  speaker_source TEXT CHECK (speaker_source IN
    ('human', 'unattributed')),
  redacted     INTEGER NOT NULL DEFAULT 0,
  content_hash TEXT NOT NULL,          -- change detection
           ONLY (§5.3)
  minted_at    TEXT NOT NULL,
  ingest_run   TEXT
);
CREATE INDEX passages_by_chapter ON passages(chapter_slug,
  t_start_s);
CREATE INDEX passages_by_path ON passages(path, line_start);
CREATE UNIQUE INDEX passages_symbol_key ON passages(path, symbol)
  WHERE symbol IS NOT NULL;

-- Append-only correction overlay. The machine layer is never
  overwritten.
CREATE TABLE corrections (
```

```

    seq      INTEGER PRIMARY KEY AUTOINCREMENT,
    pid      TEXT NOT NULL REFERENCES passages(pid),
    text     TEXT NOT NULL,
    author   TEXT NOT NULL,
    at       TEXT NOT NULL,
    reason   TEXT
);
CREATE INDEX corrections_by_pid ON corrections(pid, seq);

-- Append-only redaction log. Never a column flip in isolation.
CREATE TABLE redactions (
    seq      INTEGER PRIMARY KEY AUTOINCREMENT,
    pid      TEXT NOT NULL REFERENCES passages(pid),
    action    TEXT NOT NULL CHECK (action IN
        ('redact', 'unredact')),
    reason_code TEXT NOT NULL,
    by       TEXT NOT NULL,
    at       TEXT NOT NULL
);

-- Directed, cycle-checked relationships.
CREATE TABLE crossrefs (
    from_pid TEXT NOT NULL REFERENCES passages(pid),
    to_pid   TEXT NOT NULL REFERENCES passages(pid),
    relation TEXT NOT NULL CHECK (relation IN
        ('explains', 'demonstrates', 'continues', 'related')),
    origin   TEXT NOT NULL CHECK (origin IN
        ('derived', 'authored')),
    score    REAL,
    generation INTEGER,
    -- derived edges
    -- belong to a generation; authored do not
    PRIMARY KEY (from_pid, to_pid, relation, origin),
    CHECK (from_pid <> to_pid)
    -- self-references
    -- rejected at write time
);

-- Code-passage rename aliases. Best-effort, honestly labelled
-- (§6).
CREATE TABLE symbol_aliases (
    old_path TEXT NOT NULL, old_symbol TEXT NOT NULL,
    new_path TEXT NOT NULL, new_symbol TEXT NOT NULL,
    pid      TEXT NOT NULL REFERENCES passages(pid),
    origin   TEXT NOT NULL CHECK (origin IN
        ('detected', 'authored')),
    at       TEXT NOT NULL,
    PRIMARY KEY (old_path, old_symbol, new_path, new_symbol)
);

-- Index generations. Only a verified generation may become live
-- (D-SEARCH-5).
CREATE TABLE generations (
    generation INTEGER PRIMARY KEY,

```

```

state      TEXT NOT NULL CHECK (state IN
    ('building','verified','live','superseded','discarded')),
pid_count  INTEGER NOT NULL,
built_at   TEXT NOT NULL,
root_hash  TEXT
);
CREATE TABLE generation_members (
    generation INTEGER NOT NULL REFERENCES generations(generation),
    pid        TEXT NOT NULL REFERENCES passages(pid),
    PRIMARY KEY (generation, pid)
);

-- Lexical type-ahead. Prefix tokens sized for the measured 2-4
-- character probe workload.
CREATE VIRTUAL TABLE passages_fts USING fts5(
    pid UNINDEXED, title, body, symbol,
    prefix = '2 3 4',
    tokenize = 'unicode61 remove_diacritics 2'
);

```

Schema invariants

#	Invariant
S1	passages.pid is the only primary key in the system. Every foreign key in every table points at it.
S2	content_hash appears in no primary key, no unique index and no foreign key. Grep is not the enforcement — §5.3's behavioural gate is.
S3	crossrefs with origin = 'derived' carry a generation and are deleted wholesale when that generation is superseded. origin = 'authored' rows carry generation IS NULL and survive every re-index — they are content, not derivation.
S4	generation_members is the set that citation verification checks membership against (http-api.md §3.10 A3).
S5	A row is never DELETED from passages. Removal is redacted = 1 plus a redactions record, or a

#	Invariant
	tombstone; the identifier is never freed (M3).

5. The two survival guarantees

FR-037 exists because of exactly two operations. Each gets a named guarantee, a mechanism, and a gate with a paired mutation.

5.1 G1 — pid survives text correction (FR-038, SC-016)

Guarantee: correcting a passage’s text does not change its pid, and every citation and cross-reference pointing at it continues to resolve to *the same passage*.

Mechanism: a correction is an **append** to corrections and an update to `passages.text`. `passages.pid` is untouched. `passages.machine_text` is untouched. `content_hash` is recomputed — which is its entire job (§5.3). The anchor in the source file is untouched, because the correction edits the text *under* the anchor.

Provenance transition: `machine` → `human_corrected`, one way only. There is no path back, because the machine text is retained separately rather than overwritten.

Gate G-PID-1: create a passage, cite it, correct its text, re-run ingest and re-index; assert the pid is unchanged, the citation still resolves, `machine_text` is byte-identical to before, and `content_hash` **did** change.

Paired mutation: make ingest re-mint on `content_hash` change (i.e. reproduce the measured Lumen behaviour). The gate must FAIL. Without this mutation the gate proves nothing — and this is the precise defect the whole contract exists to prevent, so it is the one mutation that must never be skipped.

5.2 G2 — pid survives line movement (FR-037)

Guarantee: inserting, deleting or moving content **above** a passage does not change its pid. Its `source_ref.line_start/line_end` change; nothing else does.

Mechanism: the anchor travels with the text. Ingest locates passages by scanning for anchors and re-derives `source_ref` from where it found them. `source_ref` is a cache (§3.4), never a key.

Gate G-PID-2: reproduce the measured D-SEARCH-1 run 3 — prepend a section to a transcript, shifting every later section by 4 lines. Assert every `pid` is unchanged **and** every affected `source_ref` was updated to the new lines. **Paired mutation:** key ingest matching on `(path, line_start)`. The gate must FAIL.

5.3 `content_hash` — change detection, never identity

Definition

Property	Value
Algorithm	SHA-256
Input	the passage's current text, NFC-normalised, trailing whitespace stripped per line, LF-joined
Encoding	lowercase hex, prefixed <code>sha256:</code>

Permitted uses — exhaustive

1. Skip re-embedding a passage whose text has not changed since the last generation.
2. Detect that a correction actually altered the text (a no-op correction is rejected).
3. Contribute to a generation's `root_hash` for the verification gate.
4. Detect that a source file was edited outside the pipeline.

Forbidden uses — also exhaustive, and each is a defect

1. As a citation target.
2. As a cross-reference endpoint.
3. As a join key, foreign key, primary key or unique index component.
4. As an equality test for “is this the same passage”.
5. As a cache key that outlives a generation.
6. Aliased to a field named `id`, `key`, `hash`, `chunk_id` or anything else that reads as identity.

Naming is part of the contract: the field is named `content_hash` in every serialisation — JSONL, SQLite, HTTP responses, log lines — precisely so nobody mistakes it for identity. Any code treating it as identity is a defect, not a design variation.

Gate G-PID-3 (behavioural, because “*an assertion that greps a file for a string is not a test*” — constitution, Isolation by Default): change a passage’s text; assert `content_hash` changed, `pid` did not, and **every** stored citation and cross-reference to that passage still resolves to the same row. **Paired mutation**: replace the citation lookup key with `content_hash`. The gate must FAIL.

6. CODE passages — a weaker guarantee, stated openly

Text passages get identity by construction. **Code passages do not, and this contract does not pretend otherwise.**

6.1 Why it is weaker

Anchors are never written into source code (§3.3): the D2 corpus is the whole monorepo, and stamping comments into files the workshop does not own would modify unrelated submodules. Without an anchor, ingest has no travelling marker and must re-attach identity by *matching* — and matching is a heuristic.

6.2 The matching key

Component	Example
path (repo-relative)	workshop/platform/backend/ internal/search/service.go
symbol (symbol path)	internal/search. (*Service).Query

A code passage’s row is re-attached on re-ingest by (path, symbol), then by the `symbol_aliases` table.

6.3 What holds and what does not

Operation	Guarantee
Edit the body of a symbol (text correction)	Holds. Same (path, symbol); <code>pid</code> preserved; <code>content_hash</code> changes. Identical to G1.

Operation	Guarantee
Move a symbol within its file (line movement)	Holds. Position is not the key. Identical to G2.
Reformat, re-indent, re-order other symbols	Holds.
Rename a symbol	Does not hold automatically. Requires an alias record.
Move a symbol to another file	Does not hold automatically. Requires an alias record.
Delete a symbol	Row is tombstoned; the pid is never reused (M3).

6.4 The rename alias table

`symbol_aliases` maps an old (path, symbol) to a new one, preserving the pid.

origin	Source	Trust
authored	Written by a maintainer as part of the rename	Authoritative
detected	Proposed by ingest's heuristic	Best-effort, may be wrong

The detection heuristic — identical `content_hash` in the same file under a different symbol name, or an identical symbol name appearing in a different file when the old one vanished in the same run — is a **heuristic**. It can mis-attach two similar helpers. Detected aliases are recorded with `origin: "detected"` so a reviewer can see which links rest on a guess.

6.5 The guarantee actually offered, and its limit

Code passage identity is best-effort. When it fails, it fails *loudly*.

An unaliased rename causes the old pid to resolve as `not_in_registry` — an HTTP 404, a visibly broken citation. It **never** silently re-points at different code. The failure mode is a dead link a reader can report, not a confident link to the wrong thing.

That is a materially weaker promise than text passages get, and it is stated here rather than glossed:

Kind	Identity under correction	Identity under movement	Identity under rename
transcript, doc_section, diagram (inside workshop/)	Guaranteed	Guaranteed	n/a
code, and any passage outside workshop/	Guaranteed	Guaranteed	Best-effort; loud failure if unaliasd

6.6 Boundary on SC-016 / SC-016a — flagged, then closed in the spec

Status: the boundary this section first raised has been resolved. `spec.md` was amended; the recommendation below was adopted in full. The history is kept rather than deleted, because the recommendation is the reason the criterion reads the way it now does.

What this section originally said (2026-09-01, superseded). SC-016 then read *unqualified* — “100% of previously created cross-references and citations still resolve” — with no scope on which passages. Read literally as transcripts, this contract met it at 100% by construction (G1 + G2); extended to code passages under symbol rename or file move, 100% held only when an alias existed. That gap was recorded rather than assumed away, with the recommendation that it *“be resolved explicitly — either by narrowing SC-016 to the transcript scope it names, or by making an authored alias a mandatory step of any code rename inside workshop/.”*

What was adopted. Both halves. `spec.md` split the criterion in two:

Criterion	Scope	Guarantee
SC-016	passages the curriculum owns — transcripts and its own documentation	100% of cross-references and citations still resolve after correction plus re-index. Holds by construction : the identifier is anchored into the source

Criterion	Scope	Guarantee
SC-016a	source code the curriculum does not own, where no anchor can be written into the file	artifact itself (§3.1, G1, G2). Identity is keyed on symbol path with a rename alias table (§6.2, §6.4). The promise is deliberately weaker and stated rather than overclaimed: 100% of stale code references fail loudly. An unresolvable identifier returns not-found and is never silently re-pointed at different code. A rename inside the curriculum's own tree without an alias fails ingest rather than shipping a broken link.

So the first half of the recommendation became SC-016's new scoping clause, and the second half became SC-016a's ingest rule — which this contract implements for `workshop/`-owned code (an unaliased rename fails the ingest gate, exit 1, §8 R4) and, for code outside `workshop/`, converts into the loud-failure guarantee SC-016a actually claims (§6.5, mint-and-orphan).

What remains a real limit, and is not closed by the amendment.

SC-016a's guarantee is *loud failure*, not *survival*. An unaliased rename of code outside `workshop/` still breaks the citation; the contract's promise is only that the break is visible as a 404 rather than hidden behind a confident link to the wrong code. That is a materially weaker promise than §6.5's table gives text passages, and it stays stated here rather than glossed. It also inherits **P-U1** (§10): the `symbol` component of the matching key has no confirmed producer yet.

Gate G-PID-4: rename a workshop/-owned symbol with an authored alias; assert the pid is preserved and citations resolve. Then rename one **without** an alias; assert the old pid resolves as `not_in_registry` (a loud 404) and that ingest reports the unaliased rename as exit 1.
Paired mutation: make unaliased renames mint a new pid silently and re-point the old citation by nearest-text match. The gate must FAIL.

7. Resolution — how citations, cross-references and redactions resolve

Everything that points at content points at a pid and resolves through the registry. There is no second path.

7.1 The resolution function

`resolve(pid) -> Resolution`

Exactly four outcomes. Three of them are the mandated three states; the fourth is redaction, which is a determined answer of its own.

Outcome	Meaning	HTTP (http-api.md §3.8)
found	Registry readable; row present; not redacted	200
redacted	Registry readable; row present; suppressed	410 Gone
<code>not_in_registry</code>	Registry readable; row absent — a determined negative	404
undetermined	Registry unreadable, locked, or no generation is live — could not determine	503

Normative: resolution NEVER falls back to fuzzy text matching, nearest-neighbour lookup, prefix matching, or “the passage with the same `content_hash`”. An unknown pid is not “probably this one”. A

wrong-but-plausible resolution is the exact “confident but wrong” outcome the governing constitution forbids; a dead link is the correct behaviour.

undetermined is never collapsed into not_in_registry. Collapsing them would make an unreadable database look like a curriculum that never contained the passage.

7.2 Citation resolution (FR-021, SC-009)

#	Rule
C1	An answer’s citations are pids. Nothing else is citable — not line ranges, not content_hash, not Lumen chunk ids.
C2	Every citation is checked for set membership in the live generation (generation_members) before the answer is served. A deterministic set check, costing microseconds; SC-009 is unreachable without it.
C3	A citation failing C2 invalidates the whole answer (declined{no_citations}). Claims are never silently stripped to make an answer presentable.
C4	A citation resolving to redacted invalidates the whole answer (declined{redacted_evidence}).
C5	A citation resolving to undetermined yields unavailable, never declined — verification that could not run is not verification that failed.

C5 is the Honest Instruments principle applied to grounding, and it is easy to get backwards.

7.3 Redaction resolution (FR-039)

Redaction is an **append-only log** (redactions), not a column flip. The `passages.redacted` column is a materialisation of the log's latest action for that `pid`.

Mandatory propagation — all of it, not just the rendered transcript:

#	Effect
R1	Excluded from <code>passages_fts</code> and from the embedding set at generation build time. A redacted passage cannot be returned by search.
R2	<code>resolve()</code> returns <code>redacted</code> ; <code>text</code> and <code>machine_text</code> are absent from every serialisation at every layer (F6).
R3	Cross-references with a redacted endpoint are suppressed from traversal; the count is reported as <code>redacted_omitted</code> (honest that something was suppressed, without leaking what).
R4	Any stored answer whose citations intersect the redaction set is marked <code>withdrawn</code> and MUST NOT be served. Re-asking is required. Redaction propagates to stored answers, not merely to the displayed transcript.
R5	Redacting requires a rebuild of the live generation before search is honest again. Until the rebuild completes, the index reports degraded rather than serving a generation known to contain the passage.
R6	Un-redaction is an explicit action: "unredact" record — also append-only, also

#	Effect
R7	reviewable. The log is the audit trail; history is never rewritten. Publication and export are blocked until a redaction review has been recorded for the chapter (FR-039). Explicitly deciding “none required” satisfies it; skipping it does not. Enforced by pipeline-cli.md §4.6/§4.7 .

Gate G-PID-5: redact a cited passage; assert search cannot return it, resolve returns redacted (410), cross-references to it are omitted with a non-zero redacted_omitted, and the stored answer citing it is withdrawn.
Paired mutation: propagate redaction only to the rendered transcript. The gate must FAIL — demonstrating R1, R3 and R4 individually.

7.4 Cross-reference resolution (FR-018)

#	Rule
X1	Both endpoints are pids. Self-references are rejected at write time (schema CHECK).
X2	Traversal maintains a visited set; no returned path revisits a pid. Spec edge case: <i>cross-references must not create loops that trap a reader</i> .
X3	Truncation is reported (truncated_by: "cycle" \ "limit" \ "depth"), never silent.
X4	derived edges belong to a generation and are rebuilt with it. authored edges are content and survive every re-index.
X5	An edge whose target resolves undetermined makes the whole cross-reference response unavailable, not a shorter list.

8. Ingest matching algorithm (normative, deterministic)

Applied per candidate passage, in this order. The order is part of the contract: minting is the last branch (M4).

Step	Condition	Action
R1a	An anchor is present and its pid is in the registry	Attach to that row. Re-derive source_ref from the anchor's current position. Authoritative — no other check may override it.
R1b	An anchor is present, its pid is in the registry, and the same pid appears as an anchor elsewhere in the tree	Error, exit 1: duplicate anchor. A pid names one passage.
R1c	An anchor is present and its pid is not in the registry	Error, exit 1: foreign anchor. Do not mint, do not adopt. A copy-pasted block must not hijack an identity, and an unknown anchor may mean the registry was rolled back — a condition an operator must see.
R2	No anchor; kind == "code" or the path is outside workshop; (path, symbol) matches a row	Attach.
R3	No anchor; kind == "code"; (path, symbol) matches a symbol_aliases row	Attach via the alias, and record which alias was used so origin: "detected" links are reviewable.

Step	Condition	Action
R4	No anchor; kind == "code"; a symbol vanished and an unaliased candidate exists	Exit 1 for workshop/-owned code (an alias is required — §6.6). For code outside workshop/, mint a new pid and record the orphaned old one, so the broken citation is <i>loud</i> (§6.5).
R5	No anchor; text kind inside workshop/	Mint a pid; write the anchor back to the source file.
R6	No anchor; text kind outside workshop/, or a sidecar format	Mint a pid; record anchor: "sidecar" or "registry_only". Never write into the file.

8.1 Idempotency contract (FR-027, SC-011)

A second ingest over unchanged inputs MUST:

#	Assertion
I1	Mint zero pids.
I2	Write zero anchors.
I3	Produce a byte-identical <code>passages.jsonl</code> .
I4	Leave every source file byte-identical.
I5	Create no new index generation.
I6	Exit 0.

Gate G-PID-6: run ingest twice; assert I1–I6 by diffing the tree and counting mints. **Paired mutation:** make R5 unconditional (mint whenever no anchor was *read this run*). The gate must FAIL on I1, I3 and I5 simultaneously.

9. Traceability

Requirement	Where in this contract
FR-002 timestamps carried per passage	§4.2 <code>t_start_s/t_end_s</code> , F5
FR-003 uncertainty marked, never invented	§4.2 F3
FR-005 speaker attributed or stated absent	§4.2 F4
FR-016 all indexed content types are passages	§4.3 <code>kind</code> enum; §6 (code); §3.2 (diagrams)
FR-017 result → exact passage	§7.1 <code>resolve</code> + <code>source_ref</code>
FR-018 cross-references	§4.3 <code>crossrefs</code> , §7.4
FR-021 citations	§7.2
FR-027 idempotent re-ingest	§4.1, §8.1
FR-037 stable, non-positional, non-content-derived id	§1, §2, §5.1, §5.2 — the whole document
FR-038 machine layer immutable + provenance recorded	§4.2 F1/F2, §4.3 corrections, §5.1
FR-039 redaction step and propagation	§4.3 redactions, §7.3
FR-040 evidence retained with the commit	§4 — <code>passages.jsonl</code> is git-tracked and reviewable at the commit that produced it
SC-002 accuracy re-measurable later	§4.2 F1 (<code>machine_text</code> retained)
SC-009 citations resolve and are verifiable	§7.2 C2 (<code>generation_members</code> set check)
SC-013 three states distinguished	§7.1 (undetermined never collapsed)
SC-016 100% of references to OWNED passages survive correction + re-index	§5.1 G1, §5.2 G2 — 100% by construction for anchored kinds inside <code>workshop/</code>
SC-016a code identity: 100% of stale code references fail LOUDLY	§6.2 matching key, §6.4 <code>symbol_aliases</code> , §6.5 the guarantee and its limit, §6.6 scope, §8 R3/R4, gate G-PID-4 . Producer of symbol is UNVERIFIED — see P-U1 (§10).

Requirement	Where in this contract
FR-032 / SC-012 every gate has a paired mutation	G-PID-1 ... G-PID-6

9.1 Requirements deliberately not covered here

FR-001, FR-004, FR-006, FR-007, FR-026, FR-028, FR-029 → [pipeline-cli.md](#). FR-008 ... FR-015, FR-019 ... FR-025 → [http-api.md](#). FR-030, FR-031 → documentation deliverables. FR-034 → [pipeline-cli.md](#) §2.7. **FR-035 and FR-036 are covered by no contract in this set** — they are repository governance and commit-hygiene tasks, flagged in [http-api.md](#) §7.2.

10. Unverified register

#	Item	Status	Settled by
P-U1	The source of symbol for code passages. §6 keys code identity on a symbol path, but Lumen exposes no symbol table through its CLI or MCP surface (measured: no tool returns chunk identifiers or symbols), so the 58,726-symbol figure comes from its internal store, not from a supported interface.	UNVERIFIED	Either read Lumen's SQLite store directly and confirm the symbol schema is stable, or extract symbols with a Go/TypeScript parser owned by this feature. Until settled, §6's matching key has no confirmed producer.
P-U2	Anchor write-back safety on	UNVERIFIED	Round-trip a transcript with

#	Item	Status	Settled by
	generated markdown. §3.1 writes anchors into workshop/ markdown. Whether any downstream renderer, exporter or PDF step in this repository strips or re-orders HTML comments has not been tested. Registry scale. Chapter 1 is ~1,400 transcript passages; the code corpus adds tens of thousands.		anchors through the export path and diff the anchors before and after.
P-U3	passages.jsonl sort-and-rewrite on every ingest is contracted as byte-stable, but its cost at full corpus size has not been measured.	UNVERIFIED	Time a full ingest once the corpus exists. If it becomes a problem, the fix is chunked JSONL files per chapter — a layout change, not an identity change.
U3 (research.md)	That the recording contains intelligible speech. Every transcript passage in this	UNVERIFIED	The 5-minute calibration run on the extracted 300 s sample.

#	Item	Status	Settled by
	contract presumes a meaningful transcript exists.		

Measured, not assumed (and therefore not in this register): Lumen chunk ids are content-derived and position-independent (the 4b98e295c112a429 → 610e608a50f273a6 typo experiment, and the prepend experiment where all ids held while ranges moved); the recording's dual-mono audio that rules out machine speaker attribution; FTS5 availability in `modernc.org/sqlite`; and the half-written readable index observed during a live rebuild.